Android Technical Test — POS Payment App

Oct 17, 2026 · @Mostafa

Overview

Build a small Android point-of-sale app in Kotlin that takes an amount on a numpad, sends a payment to a real API, and shows whether it succeeded. You have **60 minutes**.

Work through the steps in order. Each step adds to the previous one and ends with a working app. A finished Step 2 is worth more than a half-built Step 4, so get each step working before you start the next.

Rules

- Kotlin, Android Studio, minimum SDK of your choice.
- Jetpack Compose or XML views: your choice.
- Any networking library: Retrofit, Ktor or OkHttp.
- Documentation and search are allowed.
- Commit (or tell the interviewer) when you finish each step.
- Focus on behavior. Design is not graded; a plain UI is fine.

API reference

Base URL: `https://api.dibsy.cloud`. Every request needs the header `X-Api-Key: test_key_123`. Amounts are **integers in cents**: QAR 25.50 is `2550`.

POST /pay

Starts a payment. The response comes after about 1 second, with `status: "pending"`. The final status is known about 3 seconds later through `GET /transactions/{id}`.

Headers: `X-Api-Key`, `Content-Type: application/json`, and optionally `Idempotency-Key` (8–64 characters, a UUID works). Sending the same key again returns the same transaction instead of charging twice.

```
curl -X POST https://api.dibsy.cloud/pay \
  -H "X-Api-Key: test_key_123" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: 7f3c2a10-5b1e-4c7d-9a2b-1e0f6d8c4b21" \
  -d '{"amount": 2550, "currency": "QAR"}'
```

Response `202 Accepted`:

```
{
  "id": "tr_0wdNGY4vjBfja6Cy9UkE",
  "amount": 2550,
  "currency": "QAR",
  "status": "pending",
  "reason": null,
  "createdAt": "2026-10-17T11:59:04.549Z",
  "updatedAt": "2026-10-17T11:59:04.549Z"
}
```

Send `currency` as `"QAR"` on every payment.

GET /transactions/{id}

Returns one transaction in the same shape. `status` is `pending`, `success` or `failed`.

```
curl https://api.dibsy.cloud/transactions/tr_0wdNGY4vjBfja6Cy9UkE \
  -H "X-Api-Key: test_key_123"
```

Response `200 OK`:

```
{
  "id": "tr_0wdNGY4vjBfja6Cy9UkE",
  "amount": 2550,
  "currency": "QAR",
  "status": "success",
  "reason": "Approved",
  "createdAt": "2026-10-17T11:59:04.549Z",
  "updatedAt": "2026-10-17T11:59:07.550Z"
}
```

A failed payment has `"status": "failed"` and `"reason": "Fraud detected"`.

GET /transactions?limit=20

Latest transactions first. `limit` is 1–100, default 20.

```
{ "items": [ { "id": "tr_...", "amount": 2550, "status": "success", "...":
"..." } ], "count": 1 }
```

Errors

Every error has the same body:

```
{ "error": { "code": "invalid_amount", "message": "amount must be an integer
in cents between 1 and 9999999" } }
```

HTTP	code	When
400	`invalid_amount`, `invalid_currency`, `invalid_json`, `invalid_idempotency_key`	Bad request body or header
401	`unauthorized`	Missing or wrong `X-Api-Key`
404	`not_found`	Unknown transaction id
409	`request_in_progress`	Same `Idempotency-Key` sent while the first request is still running
422	`idempotency_mismatch`	Same `Idempotency-Key` reused with a different amount
429	`rate_limited`	More than 60 requests per minute
500	`internal_error`	Server or processor failure

Test amounts

Use these to trigger each outcome:

Amount typed	Sent as	Result
QAR 25.50 (any normal amount)	`2550`	`pending`, then `success`

Amount typed	Sent as	Result
QAR 1,500.00 (over QAR 1,000.00)	`150000`	`pending`, then `failed`, "Fraud detected"
QAR 10.99 (cents end in 99)	`1099`	HTTP `500`, no transaction created
QAR 5.98 (cents end in 98)	`598`	Transaction created, but the response takes 15 seconds

Kotlin starter (optional)

You may copy this to save time.

```
// build.gradle.kts
implementation("com.squareup.retrofit2:retrofit:2.11.0")
implementation("com.squareup.retrofit2:converter-gson:2.11.0")

// AndroidManifest.xml
// <uses-permission android:name="android.permission.INTERNET" />

data class PayRequest(val amount: Long, val currency: String = "QAR")

data class Transaction(
    val id: String,
    val amount: Long,
    val currency: String,
    val status: String,   // "pending" | "success" | "failed"
    val reason: String?,
    val createdAt: String,
    val updatedAt: String,
)

data class TransactionList(val items: List<Transaction>, val count: Int)

data class ApiError(val error: ErrorBody) {
    data class ErrorBody(val code: String, val message: String)
}

interface PosApi {
    @POST("pay")
    suspend fun pay(
        @Body body: PayRequest,
```

```
        @Header("Idempotency-Key") idempotencyKey: String?,
    ): Response<Transaction>

    @GET("transactions/{id}")
    suspend fun transaction(@Path("id") id: String): Response<Transaction>

    @GET("transactions")
    suspend fun transactions(@Query("limit") limit: Int = 20):
Response<TransactionList>
}

val client = OkHttpClient.Builder()
    .addInterceptor { chain ->
        chain.proceed(
            chain.request().newBuilder()
                .header("X-Api-Key", "test_key_123")
                .build()
        )
    }
    .build()

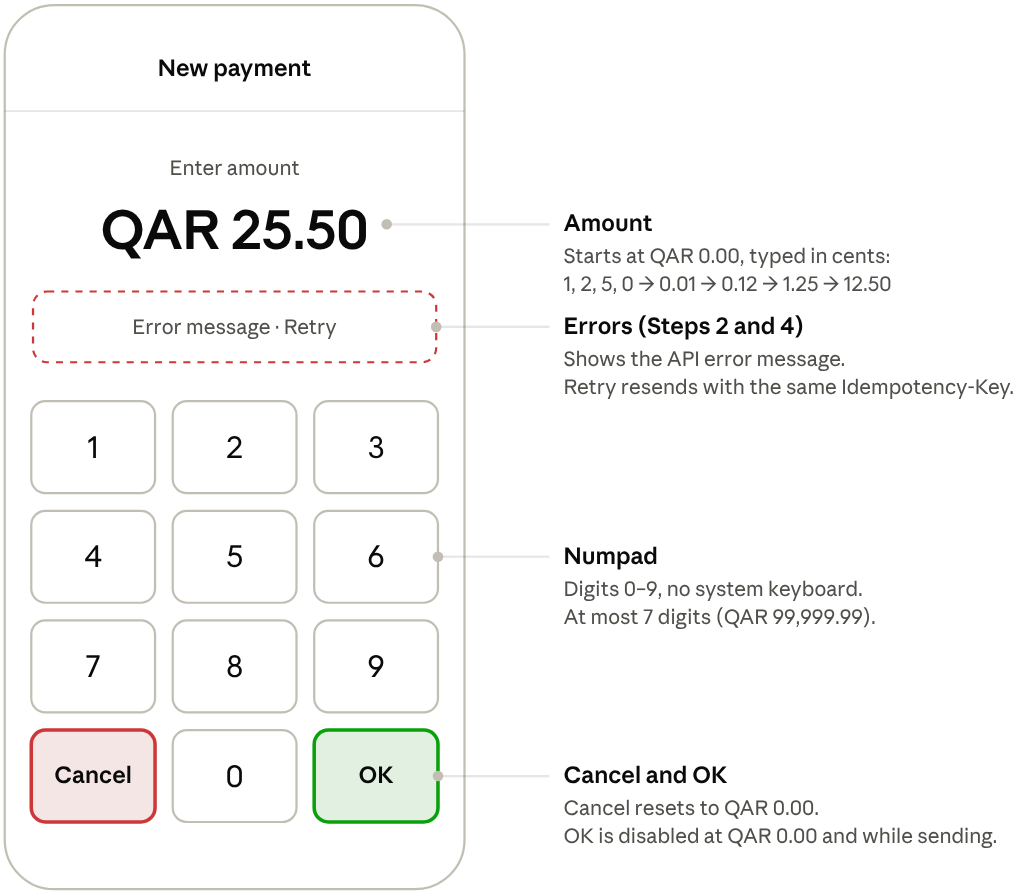
val api: PosApi = Retrofit.Builder()
    .baseUrl("https://api.dibsy.cloud/")
    .client(client)
    .addConverterFactory(GsonConverterFactory.create())
    .build()
    .create(PosApi::class.java)
```

Steps

Step 1 — Amount screen with numpad

Build the screen the cashier uses to type an amount. No networking yet.

Amount screen: type in cents, OK sends the payment



UI prototype · amount screen

A sketch, not a spec: layout and colours are up to you; the behaviour in the notes is required.

1. Show the amount at the top, starting at `QAR 0.00`.
2. Below it, a 4 × 3 grid of 12 buttons: `1 2 3`, `4 5 6`, `7 8 9`, then `Cancel`, `0`, `OK`. Do not use the system keyboard.
3. Digits are entered in cents (dirhams), like a card terminal: typing `1`, `2`, `5`, `0` shows `0.01` → `0.12` → `1.25` → `12.50`.
4. `Cancel` resets the amount to `QAR 0.00`.
5. Limit input to 7 digits (max `QAR 99,999.99`); extra digits are ignored.
6. `OK` is disabled while the amount is `QAR 0.00`.
7. Store the amount as a whole number of cents (`Long`), not a `Double`.

Done when: typing `2 5 5 0` shows `QAR 25.50`, `OK` turns on, and `Cancel` brings it back to `QAR 0.00`.

Step 2 — Send the payment

Connect `OK` to the API.

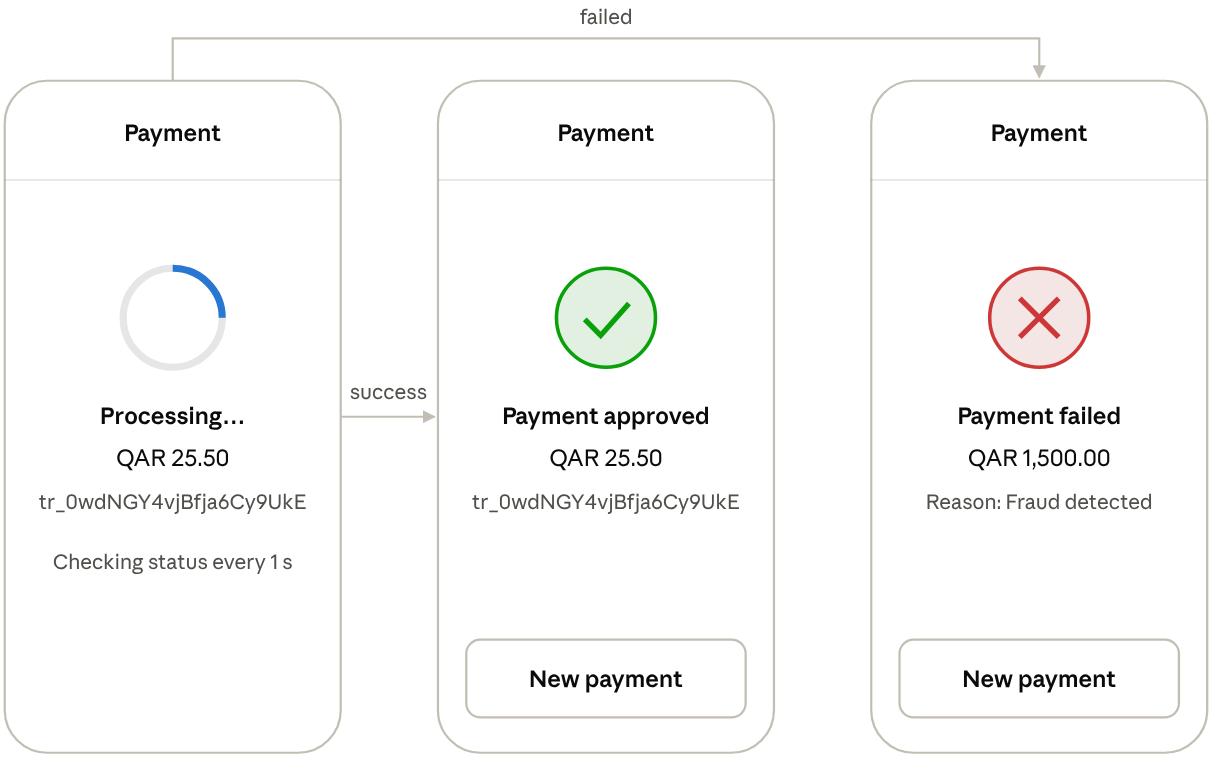
1. On `OK`, call `POST /pay` with `{"amount": <cents>, "currency": "QAR"}` and the `X-Api-Key` header.
2. While the request runs, show a loading indicator and keep `OK` disabled.
3. Make the call off the main thread (coroutines, ideally in a `ViewModel`).
4. On `202`, open a result screen that shows the transaction `id`, the amount and the status `pending`.
5. On an error (`400`, `401`, `500`, no network), stay on the amount screen and show the error `message` from the response, or a clear text for no network.

Done when: `25.50` opens the result screen with a `tr_…` id, and `10.99` shows the processor error without crashing.

Step 3 — Wait for the final result

The payment is `pending` at first. Find out how it ended.

After OK: wait for the final status, then show the result



UI prototype · processing and result screens

`New payment` goes back to the amount screen at `QAR 0.00`; pressing back from a result never sends the payment again.

1. On the result screen, call `GET /transactions/{id}` every 1 second until `status` is no longer `pending`.
2. Stop after 15 attempts and show "Still processing, check later".
3. Show a clear **Success** state with the amount, or a **Failed** state with the `reason`.
4. Add a `New payment` button that returns to the amount screen with `QAR 0.00`.
5. Pressing back on the result screen must not send the payment again.
6. Stop polling when the screen is left.

Done when: `25.50` ends on Success, and `1,500.00` ends on Failed with "Fraud detected".

Step 4 — Retries and safe double-taps

A payment app must never charge twice.

1. Generate an `Idempotency-Key` (`UUID.randomUUID()`) when `OK` is pressed, and send it with `POST /pay`.
2. Set a 10-second timeout on the HTTP client.
3. On a timeout or a `500`, show a `Retry` button. `Retry` sends the **same** amount with the **same** key.
4. A new payment (after `New payment` or `Cancel`) gets a new key.
5. Pressing `OK` quickly twice must send only one request.

Done when: `5.98` times out, `Retry` returns the transaction that was already created (same `id`), and it ends on Success.

Step 5 — Bonus (if time remains)

Pick any:

- A history screen listing `GET /transactions` (amount, status, time), newest first.
- Keep the typed amount and the result screen through a screen rotation.
- Format amounts with `NumberFormat` and the device locale.
- Arabic support: right-to-left layout and Arabic strings.
- A unit test for the cents input logic (digit, cancel, max length) or the `ViewModel`.
- Move the API key out of the source code, for example into `BuildConfig`.

Before you finish

Before time is up, even mid-step:

- [ ] The app builds and runs on the emulator or device.
- [ ] If a step is half done, comment it out or hide it so the earlier steps still work.
- [ ] Write a few lines in `NOTES.md` or tell the interviewer: which steps are done, what is missing, and what you would do next with more time.